

APELLIDOS (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NOMBRE (en MAYÚSCULAS)

--	--	--	--	--	--	--	--	--	--	--	--	--

*Cada hoja se entrega por separado y es obligatorio entregar todas las hojas. Se debe poner el **nombre completo en todas las hojas**. No se podrá entregar hasta pasados **20 minutos**. No se pueden usar hojas de sucio, para ello se dispone del espacio sobrante en cada hoja. El examen tendrá una duración máxima de **1:15 h**. No se podrá salir de los bucles usando **break o return**.*

1.- (2 pts) Escribir un programa en C que, recibiendo como parámetros los nombres de dos ficheros de texto, encripte el contenido del primer fichero grabándolo en el segundo. El primer fichero está organizado en líneas de un máximo de 120 caracteres.

La encriptación consistirá en cambiar los bits de cada carácter de las líneas teniendo en cuenta que:

- el primer carácter de la línea no cambia.
- a partir del segundo carácter, cambiará los bits de las posiciones en que en el carácter anterior estén a 1.

Por ejemplo, si los tres primeros caracteres de una línea se representan así:

01110100 00010111 11000101

al encriptarlos se guardarán así:

01110100 ---> 01100011 ---> 10100110

```
int main (int argc, char *argv[])
{ FILE *f_in, *f_out;
  char línea[121];
  int i;

  if (argc<3)
  { printf ("Error en el número de parámetros.\n");
    return (1);
  }
  if (! (f_in = fopen (argv[1], "r")))
  { printf ("Error en la apertura del fichero %s.\n", argv[1]);
    return (2);
  }
  if (! (f_out = fopen (argv[2], "w")))
  { printf ("Error en la apertura del fichero %s.\n", argv[2]);
    fclose(f_in);
    return (3);
  }

  while (fgets(línea, 121, f_in))
  if (línea[0])
  { for (i=1; línea[i]; i++)
    línea[i] ^= línea[i-1];
    fputs(línea, f_out);
  }
  fclose(f_in);
  fclose(f_out);
  return (0);
}
```

2.- (1.5 ptos) Dada la estructura definida de la siguiente forma:

```
typedef struct
{ char nombre[20], apell[50], domic[60], ciudad[25], dni[10], cod_post[6];
} USUARIO;
```

Escribir una función en C que, recibiendo como parámetros dos ficheros abiertos (el primero conteniendo usuarios como los definidos) y un código postal, grabe en el segundo todos los usuarios que tengan domicilio en el código postal pasado como parámetro.

```
void select_cpost (FILE *f_in, FILE *f_out, char *cp)
{ USUARIO aux;

while (fread(&aux, sizeof(USUARIO), 1, f_in))
if (! strcmp(aux.cod_post, cp))
fwrite (&aux, sizeof(USUARIO), 1, f_out);
}
```

3a.- (0.5 ptos) Definir, usando typedef, la estructura NODO que contendrá un USUARIO como se ha definido en el ejercicio anterior, y LISTA, que enlazará con el primer NODO de una lista dinámica.

```
typedef struct nodo
{ USUARIO user;
struct nodo * sig;
} NODO;

typedef NODO * LISTA;
```

3b.- (2 ptos) Escribir una función en C que, recibiendo como parámetros:

- un fichero abierto de USUARIOS.
- una lista dinámica de NODOs vacía.
- un apellido.

inserte en la lista todos los usuarios que tengan algún apellido igual al pasado como parámetro y devuelva como resultado cuántos nodos se han añadido a la lista.

```
unsigned insert_apell (FILE *f_in, LISTA *l, char *apellido)
{ USUARIO user;
LISTA aux;
unsigned total=0;

while (fread(&user, sizeof(USUARIO), 1, f_in))
if (strstr(user.apell, apellido))
{ aux = (LISTA) malloc (sizeof (NODO));
aux->user = user;
aux->sig = *l;
*l= aux;
total++;
}
return (total);
}
```